

```
# Lab 8: L1 Regularization - Lasso Regression
# Dataset: CO2 Emission by Vehicles

# Import necessary libraries for data manipulation, machine learning models, and evaluation metrics.
import pandas as pd # For data manipulation and analysis.
import numpy as np # For numerical operations, especially for RMSE calculation.

# For splitting data, scaling features, and different regression models.
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.metrics import r2_score, mean_squared_error
```

```
# Load the dataset from the CSV file into a pandas DataFrame.
df = pd.read_csv("CO2 Emissions_Canada.csv")

# Display the first 5 rows of the DataFrame to get a quick overview of the data.
print("First 5 rows of dataset:")
print(df.head())

# Display a concise summary of the DataFrame, including data types and non-null values.
print("\nDataset Info:")
print(df.info())
```

First 5 rows of dataset:

	Make	Model	Vehicle Class	Engine Size(L)	Cylinders	Transmission	\
0	ACURA	ILX	COMPACT	2.0	4	AS5	
1	ACURA	ILX	COMPACT	2.4	4	M6	
2	ACURA	ILX HYBRID	COMPACT	1.5	4	AV7	
3	ACURA	MDX 4WD	SUV - SMALL	3.5	6	AS6	
4	ACURA	RDX AWD	SUV - SMALL	3.5	6	AS6	

	Fuel Type	Fuel Consumption City (L/100 km)	\
0	Z	9.9	
1	Z	11.2	
2	Z	6.0	
3	Z	12.7	
4	Z	12.1	

	Fuel Consumption Hwy (L/100 km)	Fuel Consumption Comb (L/100 km)	\
0	6.7	8.5	
1	7.7	9.6	
2	5.8	5.9	
3	9.1	11.1	
4	8.7	10.6	

	Fuel Consumption Comb (mpg)	CO2 Emissions(g/km)
0	33	196
1	29	221
2	48	136
3	25	255
4	27	244

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7385 entries, 0 to 7384
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Make                                  7385 non-null   object
1   Model                                7385 non-null   object
2   Vehicle Class                        7385 non-null   object
3   Engine Size(L)                       7385 non-null   float64
4   Cylinders                            7385 non-null   int64
5   Transmission                         7385 non-null   object
6   Fuel Type                            7385 non-null   object
7   Fuel Consumption City (L/100 km)     7385 non-null   float64
8   Fuel Consumption Hwy (L/100 km)      7385 non-null   float64
9   Fuel Consumption Comb (L/100 km)     7385 non-null   float64
10  Fuel Consumption Comb (mpg)           7385 non-null   int64
11  CO2 Emissions(g/km)                  7385 non-null   int64
dtypes: float64(4), int64(3), object(5)
memory usage: 692.5+ KB
None
```

```
"""
Features:
Engine Size
Cylinders
```

cylinders

Fuel Consumption Comb (L/100 km)

Target:

CO2 Emissions (g/km)

"""

Define the features (independent variables) for the model.

X = df[["Engine Size(L)", "Cylinders", "Fuel Consumption Comb (L/100 km)"]]

Define the target variable (dependent variable) for the model.

y = df["CO2 Emissions(g/km)"]

Display a sample of the selected features.

print("\nFeature sample:")

print(X.head())

Display a sample of the selected target variable.

print("\nTarget sample:")

print(y.head())

Feature sample:

	Engine Size(L)	Cylinders	Fuel Consumption Comb (L/100 km)
0	2.0	4	8.5
1	2.4	4	9.6
2	1.5	4	5.9
3	3.5	6	11.1
4	3.5	6	10.6

Target sample:

0	196
1	221
2	136
3	255
4	244

Name: CO2 Emissions(g/km), dtype: int64

Split the dataset into training and testing sets.

X_train, y_train will be used to train the model.

X_test, y_test will be used to evaluate the model's performance.

test_size=0.2 means 20% of the data will be used for testing, and 80% for training.

random_state=42 ensures reproducibility of the split.

X_train, X_test, y_train, y_test = train_test_split(

X, y, test_size=0.2, random_state=42

)

Print the shapes of the training and testing feature sets to verify the split.

print("\nTraining shape:", X_train.shape)

print("Testing shape:", X_test.shape)

Training shape: (5908, 3)

Testing shape: (1477, 3)

"""

StandardScaler is required for Lasso and Ridge because

regularization is sensitive to feature scaling.

"""

Initialize the StandardScaler. This will scale features to have zero mean and unit variance.

scaler = StandardScaler()

Fit the scaler on the training data and transform both training and testing data.

fitting only on training data prevents data leakage from the test set.

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

Print a sample of the scaled training data to verify the transformation.

print("\nScaled training sample:")

print(X_train_scaled[:5])

Scaled training sample:

[[-0.12149345	0.20534188	-0.37681659]
[0.6141664	0.20534188	0.59257065]
[-0.12149345	0.20534188	0.24636092]
[0.17277049	0.20534188	-0.41143756]
[-0.85715331	-0.88513888	-0.79226826]]

```

"""
Train baseline Linear Regression model
"""

# Initialize the Linear Regression model.
lr_model = LinearRegression()
# Train the Linear Regression model using the scaled training data.
lr_model.fit(X_train_scaled, y_train)

# Make predictions on the scaled test set using the trained Linear Regression model.
y_pred_lr = lr_model.predict(X_test_scaled)

# Print the performance metrics for Linear Regression.
print("\n ♦ Linear Regression Performance")
# Calculate and print the R2 Score, which indicates the proportion of variance in the dependent variable that can be predicted
print("R2 Score:", r2_score(y_test, y_pred_lr))
# Calculate and print the Root Mean Squared Error (RMSE), which measures the average magnitude of the errors.
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lr)))

```

♦ Linear Regression Performance
R2 Score: 0.8773348735033225
RMSE: 20.540748085335153

```

"""
Train Lasso Regression model
"""

# Initialize the Lasso Regression model with an alpha (regularization strength) value.
# Alpha can be tuned to control the amount of regularization.
lasso_model = Lasso(alpha=0.1) # You can tune alpha
# Train the Lasso Regression model using the scaled training data.
lasso_model.fit(X_train_scaled, y_train)

# Make predictions on the scaled test set using the trained Lasso model.
y_pred_lasso = lasso_model.predict(X_test_scaled)

# Print the performance metrics for Lasso Regression.
print("\n ♦ Lasso Regression Performance")
# Calculate and print the R2 Score for Lasso.
print("R2 Score:", r2_score(y_test, y_pred_lasso))
# Calculate and print the RMSE for Lasso.
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lasso)))

# Print the coefficients learned by the Lasso model. Lasso can set some coefficients to zero, performing feature selection.
print("\nLasso Coefficients:")
print(lasso_model.coef_)

```

♦ Lasso Regression Performance
R2 Score: 0.8773150046187961
RMSE: 20.54241157861251

Lasso Coefficients:
[7.60546164 11.64853154 38.27235764]

```

"""
Use GridSearchCV to find best alpha for Ridge Regression
"""

# Initialize the Ridge Regression model.
ridge = Ridge()

# Define the parameter grid for alpha values to search over.
# Alpha is the regularization strength for Ridge Regression.
param_grid = {
    "alpha": [0.01, 0.1, 1, 10, 50, 100]
}

# Initialize GridSearchCV to find the best alpha.
# estimator: The model to tune (Ridge).
# param_grid: The dictionary of hyperparameters to search.
# cv: Number of folds for cross-validation.
# scoring: The metric to optimize (R2 score).
grid_search = GridSearchCV(
    estimator=ridge,
    param_grid=param_grid,

```

```

cv=5,
scoring="r2"
)

# Fit GridSearchCV to the scaled training data to find the best alpha.
grid_search.fit(X_train_scaled, y_train)

# Print the best alpha value found by GridSearchCV.
print("\nBest Alpha for Ridge:", grid_search.best_params_)
# Print the best cross-validation R2 score achieved with the best alpha.
print("Best CV Score:", grid_search.best_score_)

```

```

Best Alpha for Ridge: {'alpha': 10}
Best CV Score: 0.8789384715475002

```

```

"""
Train Ridge model using best alpha
"""

# Get the best Ridge model (with the optimal alpha) from the GridSearchCV results.
best_ridge = grid_search.best_estimator_

# Make predictions on the scaled test set using the best Ridge model.
y_pred_ridge = best_ridge.predict(X_test_scaled)

# Print the performance metrics for the Ridge Regression model with the best alpha.
print("\n ♦ Ridge Regression Performance (Best Alpha)")
# Calculate and print the R2 Score for the best Ridge model.
print("R2 Score:", r2_score(y_test, y_pred_ridge))
# Calculate and print the RMSE for the best Ridge model.
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_ridge)))

```

```

♦ Ridge Regression Performance (Best Alpha)
R2 Score: 0.8773271303605337
RMSE: 20.541396384652405

```

```

"""
Compare all models
"""

# Create a pandas DataFrame to compile and display the performance metrics of all trained models.
results = pd.DataFrame({
    "Model": ["Linear Regression", "Lasso Regression", "Ridge Regression"],
    "R2 Score": [
        r2_score(y_test, y_pred_lr),
        r2_score(y_test, y_pred_lasso),
        r2_score(y_test, y_pred_ridge)
    ],
    "RMSE": [
        np.sqrt(mean_squared_error(y_test, y_pred_lr)),
        np.sqrt(mean_squared_error(y_test, y_pred_lasso)),
        np.sqrt(mean_squared_error(y_test, y_pred_ridge))
    ]
})

# Print the model comparison table.
print("\n 📊 Model Comparison:")
print(results)

```

```

📊 Model Comparison:
   Model  R2 Score  RMSE
0  Linear Regression  0.877335  20.540748
1  Lasso Regression  0.877315  20.542412
2  Ridge Regression  0.877327  20.541396

```

